```
#!/usr/bin/env python3
"""
RØMANS — "Classical Music for Sufferers of ADHD" medley builder.

Reads cuesheet.json, extracts every excerpt from sources/, loudness-matches
them (static gain only — internal dynamics like the Haydn surprise are
preserved), builds the Bolero snare bed under Pomp & Circumstance as a true
overlay, chains everything with per-joint transitions (hard cut / crossfade /
silence), and renders:

    medley_draft.wav      full-quality master
    medley_draft.mp3      320k listening copy
    timeline.txt          cumulative timestamps per excerpt (orchestrator map)

Modes:
    python3 build_medley.py               check sources, build everything
    python3 build_medley.py --check-only  just validate sources + timestamps
    python3 build_medley.py --audit       also render every joint as a short
                                          clip into joints/ for fast checking
    python3 build_medley.py --stems       also export each excerpt to stems/
    python3 build_medley.py --skip-normalize   fast draft, no loudness match

Requires: python3 (stdlib only) and ffmpeg/ffprobe on PATH.
"""

import argparse
import glob
import json
import os
import re
import shutil
import subprocess
import sys

ROOT = os.path.dirname(os.path.abspath(__file__))
TMP = os.path.join(ROOT, "build_tmp")
FFMPEG = "ffmpeg"
FFPROBE = "ffprobe"

EDGE_FADE = 0.012   # seconds; click-guard on every splice edge
CUT_D = 0.012       # "hard cut" is a 12 ms equal-gain blend (inaudible, clickles


# ---------------------------------------------------------------- helpers
```

```
def run(cmd):
    p = subprocess.run(cmd, capture_output=True, text=True)
    if p.returncode != 0:
        sys.exit("ffmpeg failed:\n" + " ".join(cmd) + "\n\n" + p.stderr[-2000:])
    return p


def probe_duration(path):
    p = run([FFPROBE, "-v", "error", "-show_entries", "format=duration",
             "-of", "default=nw=1:nk=1", path])
    return float(p.stdout.strip())


def find_source(pattern):
    hits = sorted(glob.glob(os.path.join(ROOT, pattern)))
    hits = [h for h in hits if os.path.splitext(h)[1].lower() in
            (".mp3", ".wav", ".flac", ".m4a", ".ogg", ".aac", ".aif", ".aiff")]
    return hits[0] if hits else None


def measure(path):
    """Integrated loudness (LUFS) + true peak (dBTP) via loudnorm analysis."""
    p = subprocess.run(
        [FFMPEG, "-hide_banner", "-nostats", "-i", path,
         "-af", "loudnorm=I=-16:TP=-1.5:LRA=11:print_format=json",
         "-f", "null", "-"],
        capture_output=True, text=True)
    m = re.search(r"\{[\s\S]*\}", p.stderr)
    if not m:
        sys.exit("Could not measure loudness for " + path)
    d = json.loads(m.group(0))
    return float(d["input_i"]), float(d["input_tp"])


def mmss(t):
    return f"{int(t // 60)}:{int(t % 60):02d}"


# ---------------------------------------------------------------- extraction

def extract_segment(src, start, dur, out):
    """Cut [start, start+dur] to 44.1k stereo wav with click-guard fades."""
    fades = (f"afade=t=in:st=0:d={EDGE_FADE},"
             f"afade=t=out:st={max(dur - EDGE_FADE, 0):.3f}:d={EDGE_FADE}")
    run([FFMPEG, "-y", "-hide_banner", "-nostats",
         "-ss", f"{start:.3f}", "-t", f"{dur:.3f}", "-i", src,
```

```
          "-af", fades, "-ac", "2", "-ar", "44100",
          "-c:a", "pcm_s16le", out])


def resolve_start(seg, file_dur):
    if "from_end" in seg:
        return max(file_dur - float(seg["from_end"]), 0.0)
    return float(seg.get("start", 0))


def render_item(item, cfg, normalize=True):
    """Extract (multi-)segments, loudness-match, apply bed overlay. -> wav path"""
    iid = item["id"]
    src = find_source(item["file"])
    file_dur = probe_duration(src)

    seg_paths = []
    for k, seg in enumerate(item["segments"]):
        start = resolve_start(seg, file_dur)
        sp = os.path.join(TMP, f"{iid}_seg{k}.wav")
        extract_segment(src, start, float(seg["duration"]), sp)
        seg_paths.append(sp)

    raw = os.path.join(TMP, f"{iid}_raw.wav")
    if len(seg_paths) == 1:
        shutil.copyfile(seg_paths[0], raw)
    else:  # internal splice (e.g. Mountain King creep -> prestissimo)
        lst = os.path.join(TMP, f"{iid}_list.txt")
        with open(lst, "w") as f:
            for sp in seg_paths:
                f.write(f"file '{sp}'\n")
        run([FFMPEG, "-y", "-hide_banner", "-nostats", "-f", "concat",
             "-safe", "0", "-i", lst, "-c", "copy", raw])

    # ---- loudness match: STATIC gain only, so internal dynamics survive
    norm = os.path.join(TMP, f"{iid}_norm.wav")
    if normalize:
        lufs, tp = measure(raw)
        target = cfg["loudness_target_lufs"] + float(item.get("target_offset_db",
        tp_ceiling = cfg["true_peak_db"]
        if item.get("norm") == "peak":
            gain = tp_ceiling - tp          # peak-anchor (Haydn pp->bang, 1812 c
        else:
            gain = min(target - lufs, tp_ceiling - tp)
        clamped = max(min(gain, 30.0), -40.0)
        if clamped != gain:
            print(f"       note: {iid} gain clamped ({gain:+.1f} -> {clamped:+.1f
```

```
                f"- source level is unusual, worth a listen")
        gain = clamped
        run([FFMPEG, "-y", "-hide_banner", "-nostats", "-i", raw,
             "-af", f"volume={gain:.2f}dB", "-c:a", "pcm_s16le", norm])
    else:
        shutil.copyfile(raw, norm)

    # ---- Bolero snare bed (true overlay under this item)
    if "bed" in item:
        norm = apply_bed(item, norm, cfg, normalize)
    return norm


def apply_bed(item, main_wav, cfg, normalize):
    bed = item["bed"]
    iid = item["id"]
    bsrc = find_source(bed["file"])
    bdur = probe_duration(bsrc)
    seg = bed["segment"]

    bseg = os.path.join(TMP, f"{iid}_bedseg.wav")
    extract_segment(bsrc, resolve_start(seg, bdur), float(seg["duration"]), bseg)

    if normalize:
        lufs, tp = measure(bseg)
        gain = min(cfg["loudness_target_lufs"] - lufs, cfg["true_peak_db"] - tp)
    else:
        gain = 0.0
    gain += float(bed.get("gain_db", 0))

    solo, under, fout = float(bed["solo"]), float(bed["under"]), float(bed["fade_
    total = solo + under + fout

    bloop = os.path.join(TMP, f"{iid}_bedloop.wav")
    run([FFMPEG, "-y", "-hide_banner", "-nostats",
         "-stream_loop", "-1", "-i", bseg, "-t", f"{total:.3f}",
         "-af", (f"volume={gain:.2f}dB,"
                 f"afade=t=in:st=0:d={solo},"
                 f"afade=t=out:st={solo + under:.3f}:d={fout}"),
         "-c:a", "pcm_s16le", bloop])

    mixed = os.path.join(TMP, f"{iid}_mixed.wav")
    delay_ms = int(solo * 1000)
    run([FFMPEG, "-y", "-hide_banner", "-nostats",
         "-i", main_wav, "-i", bloop,
         "-filter_complex",
         (f"[0:a]adelay={delay_ms}:all=1[m];"
```

```
          f"[m][1:a]amix=inputs=2:duration=longest:normalize=0,"
          f"alimiter=limit=0.95[out]"),
         "-map", "[out]", "-c:a", "pcm_s16le", mixed])
    return mixed


# ---------------------------------------------------------------- assembly

def make_silence(gap, idx):
    p = os.path.join(TMP, f"sil_{idx}.wav")
    run([FFMPEG, "-y", "-hide_banner", "-nostats",
         "-f", "lavfi", "-i", "anullsrc=r=44100:cl=stereo",
         "-t", f"{gap:.3f}", "-c:a", "pcm_s16le", p])
    return p


def build_parts(cue, normalize):
    """-> list of {path, dur, joint:(d, curve), item|None}"""
    parts = []
    for i, item in enumerate(cue["items"]):
        tr = item["transition_in"]
        ttype = tr["type"]
        print(f"  [{i + 1:2d}/{len(cue['items'])}] {item['id']}")
        if ttype == "silence" and parts:
            sp = make_silence(float(tr["gap"]), i)
            parts.append({"path": sp, "dur": probe_duration(sp),
                          "joint": (CUT_D, "tri"), "item": None})
            joint = (CUT_D, "tri")
        elif ttype == "crossfade":
            joint = (float(tr["duration"]), "qsin")
        else:  # cut / start
            joint = (CUT_D, "tri")
        wav = render_item(item, cue, normalize)
        parts.append({"path": wav, "dur": probe_duration(wav),
                      "joint": joint, "item": item})
    return parts


def chain(parts, out_wav):
    cmd = [FFMPEG, "-y", "-hide_banner", "-nostats"]
    for p in parts:
        cmd += ["-i", p["path"]]
    if len(parts) == 1:
        run(cmd + ["-c:a", "pcm_s16le", out_wav])
        return
    fg, prev = [], "[0:a]"
    for i in range(1, len(parts)):
```

```
        d, curve = parts[i]["joint"]
        d = min(d, parts[i - 1]["dur"] - 0.005, parts[i]["dur"] - 0.005)
        lbl = f"[m{i}]"
        fg.append(f"{prev}[{i}:a]acrossfade=d={d:.3f}:c1={curve}:c2={curve}{lbl}"
        prev = lbl
    cmd += ["-filter_complex", ";".join(fg), "-map", prev,
            "-c:a", "pcm_s16le", out_wav]
    run(cmd)


def write_timeline(parts, path):
    t, act, lines = 0.0, None, []
    for i, p in enumerate(parts):
        if i > 0:
            t -= p["joint"][0]
        if p["item"]:
            it = p["item"]
            if it.get("act") != act:
                act = it.get("act")
                lines.append(f"\n== {act} ==")
            flag = "" if it.get("verified", True) else "   << ESTIMATE - audit"
            lines.append(f"{mmss(t):>6}  {it['id']:<20} ({p['dur']:5.1f}s)  "
                         f"{it['piece']}{flag}")
        t += p["dur"]
    lines.append(f"\nTOTAL: {mmss(t)}  ({t:.1f}s)")
    txt = "\n".join(lines)
    with open(path, "w") as f:
        f.write(txt + "\n")
    print(txt)


def render_joints(parts, outdir):
    """Every musical joint as a ~10s clip: tail of prev + transition + head of ne
    os.makedirs(outdir, exist_ok=True)
    n = 0
    for i in range(1, len(parts)):
        if parts[i]["item"] is None:
            continue
        # walk back over any silence part to the previous music
        j = i - 1
        sil = None
        if parts[j]["item"] is None:
            sil = parts[j]
            j -= 1
        prev, cur = parts[j], parts[i]
        tail = os.path.join(TMP, f"jt_{i}_tail.wav")
        head = os.path.join(TMP, f"jt_{i}_head.wav")
```

```
        run([FFMPEG, "-y", "-hide_banner", "-nostats", "-sseof", "-4",
             "-i", prev["path"], "-c:a", "pcm_s16le", tail])
        run([FFMPEG, "-y", "-hide_banner", "-nostats", "-t", "4",
             "-i", cur["path"], "-c:a", "pcm_s16le", head])
        seq = [{"path": tail, "dur": probe_duration(tail),
                "joint": (CUT_D, "tri"), "item": None}]
        if sil is not None:
            seq.append(dict(sil))
        seq.append({"path": head, "dur": probe_duration(head),
                    "joint": cur["joint"], "item": None})
        n += 1
        chain(seq, os.path.join(outdir, f"{n:02d}_into_{cur['item']['id']}.wav"))
    print(f"Rendered {n} joint previews -> {outdir}/")


# ------------------------------------------------------------------ checking

def check(cue):
    ok = True
    print("Checking sources...")
    seen = {}
    for item in cue["items"]:
        pats = [(item["id"], item["file"])]
        if "bed" in item:
            pats.append((item["id"] + " (bed)", item["bed"]["file"]))
        for label, pat in pats:
            src = find_source(pat)
            if not src:
                print(f"  MISSING  {label}: no file matches {pat}")
                ok = False
                continue
            if src not in seen:
                seen[src] = probe_duration(src)
            fdur = seen[src]
            segs = item["segments"] if "(bed)" not in label else [item["bed"]["se
            for seg in segs:
                start = resolve_start(seg, fdur)
                need = start + float(seg["duration"])
                if need > fdur + 0.5:
                    print(f"  RANGE    {label}: needs up to {need:.0f}s but "
                          f"{os.path.basename(src)} is {fdur:.0f}s")
                    ok = False
    unv = [i["id"] for i in cue["items"] if not i.get("verified", True)]
    if unv:
        print("\nTimestamps to verify by ear (run --audit):")
        for u in unv:
            print(f"  - {u}")
```

```
    print("Source check " + ("PASSED" if ok else "FAILED") + ".\n")
    return ok


# ---------------------------------------------------------------- main

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--check-only", action="store_true")
    ap.add_argument("--audit", action="store_true")
    ap.add_argument("--stems", action="store_true")
    ap.add_argument("--skip-normalize", action="store_true")
    args = ap.parse_args()

    with open(os.path.join(ROOT, "cuesheet.json")) as f:
        cue = json.load(f)

    if not check(cue):
        sys.exit(1)
    if args.check_only:
        return

    os.makedirs(TMP, exist_ok=True)
    print("Rendering excerpts...")
    parts = build_parts(cue, normalize=not args.skip_normalize)

    base = os.path.join(ROOT, cue.get("output_basename", "medley_draft"))
    print("Assembling chain...")
    chain(parts, base + ".wav")
    run([FFMPEG, "-y", "-hide_banner", "-nostats", "-i", base + ".wav",
         "-c:a", "libmp3lame", "-b:a", "320k", base + ".mp3"])
    write_timeline(parts, os.path.join(ROOT, "timeline.txt"))

    if args.stems:
        sd = os.path.join(ROOT, "stems")
        os.makedirs(sd, exist_ok=True)
        for p in parts:
            if p["item"]:
                shutil.copyfile(p["path"],
                                os.path.join(sd, p["item"]["id"] + ".wav"))
        print(f"Stems -> {sd}/")
    if args.audit:
        render_joints(parts, os.path.join(ROOT, "joints"))

    print(f"\nDone: {base}.mp3")
```

```
if __name__ == "__main__":
    main()
```